YENEPOYA INSTITUTE OF ARTS,SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENTUNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

LOW LEVEL DESIGN

ON

OPTIMIZATION STRATEGIES FOR NATURAL LANGUAGE INTERFACE FOR EDTECH INFRASTRUCTURE

SUBMITTED BY

SWALIH BIN HAMEED

REGNO :

23BCAICD105

LH-06

GUIDED BY

MISS DHEEKSHA SHETTY

SUBMISSION DATE

25-04-2026

Table of Contents

1. Introduction

1.1 Scope of the Document
1.2 Intended Audience
1.3 System Overview

2. System Design

2.1 Application Design
2.2 Process Flow
2.3 Information Flow
2.4 Components Design
2.5 Key Design Considerations

3. Data Design

3.1 Data Model
3.2 Data Access Mechanism
3.3 Data Retention Policies
3.4 Data Migration

4. Interfaces

5. State and Session

Management 6. Caching

7. Non-Functional Requirements

7.1 Security Aspects
7.2 Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

Definestheinternalstructureandlogic of a CLI-based chatbot using Python with basic string matching to handle EdTech-related queries.

1.2 Intended Audience

- Developers
- Students implementing the project
- Reviewers/evaluators

1.3 System Overview

Acommand-linechatbotthat:

- Accepts user input
- Matches keywords using basic string logic
- Returns predefined responses related to EdTech systems

2. System Design

2.1 Application Design

Architecture Type: Monolithic CLI Application

Layers:

- Input Layer (CLI)
- Processing Layer (Parser + Matcher)
- Response Layer (Output Generator)

2.2 Process Flow

1. User enters command in CLI
2. Input is normalized (lowercase, trimmed)
3. Keyword matching is performed
4. Corresponding response is fetched
5. Response is displayed
6. Loop continues until exit command

2.3 Information Flow

```
UserInput → Input Handler → Text Normalizer → Keyword Matcher  →
Response Fetcher → CLI Output
```

2.4 Components Design

1. CLI Handler (cli.py)

- Responsible for:
  - Taking user input
  - Running loop
  - Exit handling

Functions:

```
def start_chat():
def get_user_input():
def display_output(response):
```

2. Input Processor (`processor.py`)

- Cleans and prepares input

Functions:

```
def normalize_text(text):
    # lowercase, strip spaces
```

3. Keyword Matcher (matcher.py)

- Matches user input with predefined keywords

Approach:

- if/elif OR dictionary-based matching

Functions:

```
def match_query(text):
```

4. Response Manager(`responses.py`)

- Stores predefined responses

Structure:

```
responses = {
    "help": "Available commands: help, status, courses",
    "status": "All systems operational",
    "courses": "Available courses: AI, ML, DevOps"
}
```

5. Logger (Optional - logger.py)

- Logs user queries for debugging

2.5 Key Design Considerations

- Keep logic simple (basic string matching)

- Modular code for easy extension
- Avoid over-engineering (no NLP libraries)
- Easy to add new commands

3. Data Design

3.1 Data Model

Simple in-memory structure:

```
{
    "keyword": "response"
}
```

No database required.

3.2 Data Access Mechanism

- Direct dictionary lookup
- In-memory access (O(1))

3.3 Data Retention Policies

- No persistent storage
- Optional logging can be stored in text files

3.4 Data Migration

- Not applicable (no DB)

4. Interfaces

User Interface

- Command Line Interface (CLI)

Input Format

```
> help
> show status
> courses
```

Output Format

```
Bot:Availablecommands...
```

5. State and Session Management

- Statelesssystem
- Each query processed independently
- Session ends on exit command

6. Caching

- Notrequired
- Optional: cache last response for repeated queries

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data handling
- Input sanitization (basic)

- Prevent command injection (no system calls)

7.2 Performance Aspects

- Response time: < 100ms
- Lightweight execution
- Works offline

8. References

- Python Documentation
- Basic String Matching Techniques
- CLI Application Design Principles